Future Generation Computer Systems 124 (2021) 390–405

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

LogNADS: Network anomaly detection scheme based on log semantics representation

Xu Liu [a,b], Weiyou Liu [a], Xiaoqiang Di [a,b,c,*], Jinqing Li [a,b], Binbin Cai [c], Weiwu Ren [a], Huamin Yang [a,b]

[a] School of Computer Science and Technology, Changchun University of Science and Technology, Changchun, China
[b] Jilin Province Key Laboratory of Network and Information Security, Changchun, China
[c] Information Center, Changchun University of Science and Technology, Changchun, 130022, China

ARTICLE INFO

Article history:
Received 16 December 2020
Received in revised form 15 April 2021
Accepted 17 May 2021
Available online 24 May 2021

Keywords:
Anomaly detection
Log
Semantics representation
LSTM

ABSTRACT

Semantics-aware anomaly detection based on log has attracted much attention. However, the existing methods based on the weighted aggregation of all word vectors might lose the semantic relationship of word order and cannot maintain the unique representation, and the methods based on word order-preserving by concatenating all word vectors might lead to a high computation time cost. To solve these issues and further improve the sequential anomaly detection, this paper proposes a network anomaly detection scheme LogNADS by designing a novel log semantics representation method and an adaptive sequence data construction method. It first discards the useless words and then selects theme words to hold the log abstraction and maintain a low time cost as well. Subsequently, it concatenates theme words' vectors based on the original word order to maintain the unique representation and avoid the word order loss. Furthermore, to better detect the sequential anomalies, we utilize the sliding window scheme and design a method to compute the optimal window size for constructing the log sequence self-adaptively, and then LSTM is built to extract timing characteristics of the log sequences. Experimental results conducted on the public benchmark HDFS dataset and BGL dataset demonstrate the effectiveness of LogNADS through comparing with other state-of-the-art methods in the detection accuracy and time cost. Moreover, the statistical significance tests prove the superior performance.
© 2021 Elsevier B.V. All rights reserved.

1. Introduction

Today's network systems or services are becoming increasingly complex. To maintain the consecutive service has become essential [1]. Any anomalous behaviors, such as frequent authentication verification failures, will increase the risk. Therefore, detecting anomalies plays an important role in network security maintenance [2], especially for the sequential anomaly which contains many steps, such as Advanced Persistent Threat (APT). Log is used to record runtime status including the information regarding the activities within the system, generated by either a user, an application or the system itself [3]. The semantic information of massive log data provides the security operating center with some assistance to detect anomalies [4–6]. But log is a typical type of natural text language that is unstructured and needs human to analyze manually. How to extract the semantic information from massive logs, and then construct the sequence data to train a good anomaly detection model instead of security expert's manual analysis is necessary. According to the existing log-related research, how to transform the log data into vectors, upon satisfying lossless semantics and low computation time cost is still the problem to be solved for improving the network anomaly detection [4,7].

Existing log sequence-based anomaly detection methods can be roughly into two categories: log message counter-based methods [8,9] and distributed representation-based methods. To represent the raw log messages as vectors, each raw log message is first parsed into a constant template and dynamic variables (such as IP). Using template index to perform anomaly detection [8,9] might induce false alarms since the indexes do not consider the semantics of different templates [4]. On the contrary, the distributed representation methods vectorize the log template by summing all words' vectors based on their weights [10]. Though they maintain the same size of different log template vectors, they cannot maintain unique semantics for the distinct template since the word order information is lost in the process of aggregation [11,12]. For example, two different templates "storage interrupt caused by input errors" and "input errors caused by storage

* Correspondence to: Changchun University of Science and Technology, Weixing Road NO.7089, Chaoyang District, Changchun, Jilin Province, China.
E-mail address: dixiaoqiang@cust.edu.cn (X. Di).

https://doi.org/10.1016/j.future.2021.05.024
0167-739X/© 2021 Elsevier B.V. All rights reserved.

interrupt" obtain the same template vector after weighted aggregating. The same phenomenon might occur in other cases, especially when the normal template and the anomaly template are represented as the same vector that will be used to train detection model with different labels, which will confuse the detection model. So an accurate representation of the log template semantics will be helpful for avoiding the ambiguity of represented vectors and further contributes to improving the detection accuracy of various detection models. For this reason, some word order-preserving methods concatenate all word vectors in order and set all template vectors' dimension same as that of the longest template [12,13]. Though this kind of methods preserve the word order, the obtained template vector's dimension is so big that it will increase the computation time cost [14]. Consequently, how to balance the trade-off between the semantics representation and computation time is considered in this paper.

A novel network anomaly detection scheme based on log semantics representation, LogNADS, is proposed. Firstly, we implement a log parsing algorithm [15] to transform raw log messages into templates and variables. To extract the semantic features of templates, we design a theme word order-preserving method by selecting theme words to present the template semantics. Afterwards, to maintain the accurate and unique template representation, the template vector is represented as a low-dimensional vector by concatenating all the theme words' vectors. Through preserving the order of extracted theme words, the log abstraction is represented, and also the unique representation of distinct template is guaranteed. Besides, to ensure the same size of different template vectors, we empirically set the number of extracted theme words instead of the length of the longest template, and a low-dimensional vector for each word is embedded to finally help reduce the computation time cost. To construct the sequence data, the existing sequence construction is usually performed by the fixed window or the slicing window whose window size is determined by assumption or based on the knowledge, which is not adaptable to different scenarios. We utilize the sliding window scheme and design a window size computation method according to the history data distribution and information entropy instead of manually setting the sequence length, and it can be implemented in different scenarios to promote the sequence anomaly detection. Finally, Long Short-Term Memory (LSTM) is built for extracting the temporal characteristics of sequence data.

We have evaluated LogNADS on the public benchmark big data log datasets: HDFS [16] is collected from the distributed system and BGL [17] is collected from the supercomputer. The experimental results suggest that LogNADS outperforms other state-of-the-art methods in the perspectives of detection accuracy and time performance. The primary contributions are listed as follows:

(1) A novel semantics representation method is proposed. It keeps the log abstraction and the semantic relationship between the word order, and also guarantees the unique representation of distinct log template. An accurate representation of the log template semantics will be helpful for avoiding the ambiguity of represented vectors and further contributes to improving the detection accuracy of various detection models.

(2) An automatic log sequence vector construction method is designed. It computes the sliding window size automatically according to the history data distribution and then constructs sequence vector in each sliding window. The automatic sequence construction method can be implemented in many scenarios to promote the sequence anomaly detection.

(3) A detection scheme LogNADS is constructed and proved effective on the benchmark big data log datasets, HDFS and BGL. It outperforms state-of-the-art methods in the perspectives of detection accuracy and time performance.

The paper is organized as follows: Section 2 introduces the background and related works of network anomaly detection. Section 3 details the problem statement. Section 4 describes the proposed LogNADS. Section 5 presents our experimental results. Section 6 discusses the possible threats to validity. Section 7 states the conclusion and future work.

2. Background and related work

In this section, we introduce the background of log-based anomaly detection and some log-related terms first. Then, we analyze the representation-related work of network anomaly detection to better illustrate the motivation and focus of this paper.

2.1. Background

In the Concise Oxford Dictionary[1], 'anomaly' is defined as "something that deviates from what is standard, normal, or expected". Generally, network anomaly detection can be referred to the problem of finding objects in a dataset that do not conform to the well defined notions of expected behaviors [18], or identifying the pattern suggested by the majority of observations, and then considering the observations that do not follow the assumed pattern as anomalies [19], and other types depending on the definition they adopt.

The network anomaly detection scheme can be implemented in many scenarios based on different types of data including network traffic, network connection feature, log data and so on. Entropy-based [20] and ensemble learning algorithm-based [21] anomaly detection methods have been explored using network traffic data. Feature clustering-based [22], local feature engineering-based [23] and probabilistic-driven ensemble approach-based [24] anomaly detection methods have been explored using connection features. A multi-stage processing method has been proposed to improve DBSCAN-based network anomaly detection accuracy [25]. Not only these methods but also many other representation methods performed well in the anomaly event detection, but they did not consider the sequence anomaly that includes many events or operations. For example, the brute force attack or APT attack usually includes many continuous operations in time. The similar attack or anomalous behaviors are called sequence anomaly in this paper.

Log data contains timing characteristics, so many researchers have tried to utilize log data to detect the sequence anomaly. Since recurrent neural network (RNN) models show advantage over other methods on learning the timing characteristics of sequential data [26], they have been widely used. Specially, as the typical RNN, Long Short-Term Memory (LSTM) [27] and Bidirectional-LSTM (bi-LSTM) [28] have performed promising ability in detecting the sequence anomaly [4,29–31]. Inspired by these work, LSTM is constructed in this paper to detect the log-based sequential anomaly. But the LSTM cannot learn the knowledge from the raw unstructured logs, so we focus on extracting semantics from logs and then representing the semantic information as vectors accepted by LSTM.

2.2. Preliminaries

Generally, a log file contains many lines of data, and every line can be seen as a log message [32]. To take use of the knowledge of log messages, the unstructured log messages need to be processed for further analysis. Take the HDFS log as an example. As shown in Fig. 1, there are five fields in each line, the last field is called as event in which the action details are recorded. Normally,

[1] Concise Oxford Dictionary — https://en.oxforddictionaries.com.

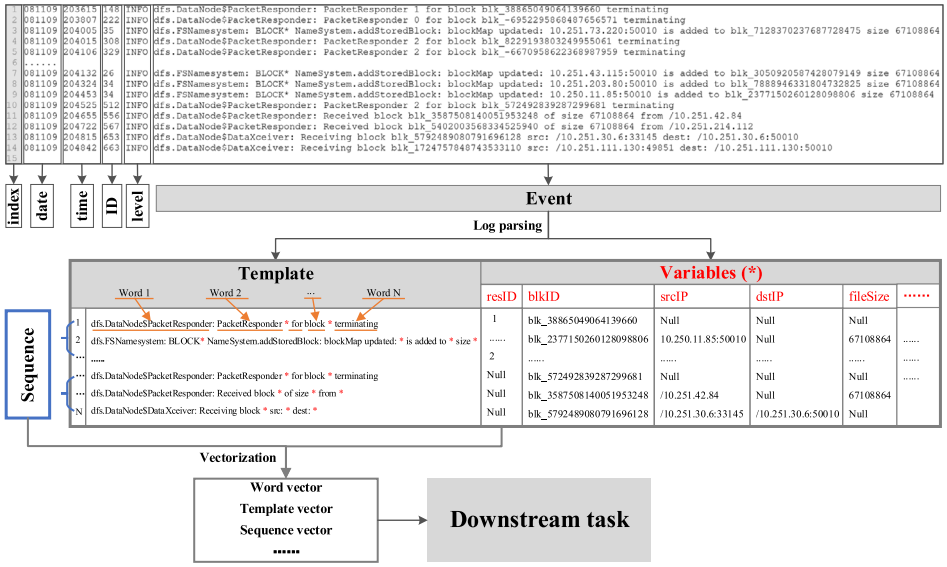


Fig. 1. The processing of log data.

the events vary with different operations, but they have the similar characteristic. The events are consisted of a constant template and variables. Variables might be number (e.g. size), character (e.g. path) or the other types, and template is composed of many words separated by the space. Log parsing [33] aims to extract the variables from the events and then keep the remaining constant parts as template. In some scenarios, such as APT detection, every APT attack includes more than one operation. In this case, the sequence is represented as many continuous templates in time. With the help of natural language processing (NLP) technologies, word, template and sequence will be transformed into vectors for the downstream tasks [34]. For the same object, different representation methods obtain different vectors that will go on influencing the detection accuracy and time performance. How to design an efficient log representation method has become the research focus of recent studies.

2.3. Related work

We introduce the log representation-related work of network anomaly detection from two views: the whole representation and the distributed representation. The whole representation methods take the log template as a whole, and the basic processing unit of log representation is the template. On the contrary, the distributed representation methods take the log template as a list of words or phrases, and the basic processing unit is the phrase or word or even the character.

The whole representation method can be realized by extracting the statistical features or using log template index to compute the log message counter in a period of time. For example, PCA (Principal Component Analysis) [8], Invariant Mining [9] and DeepLog [29] have utilized log template index counter to capture quantitative anomalies. The log count vectors far from the normal space or violate the invariants are detected as anomalies. LogEvent2vec [35] and CausalConvLSTM [36] encode each unique template first by One-Hot encoder, and then each log template vector has only one value 1 and the rest are all zero. Compared with the methods using index directly, the one-hot vectors treat equally on templates of different semantics since the distance of any two one-hot vectors is 1. The whole representation method can also be realized using NLP tools, such as word2vec and paragraph vector [37]. Each log template is seen as one word and the set of many template sequences is seen as a paragraph or document that is vectorized by Paragraph Vector [38]. The detection models take the represented log template vectors as input to learn the spatial characteristics between templates. However, valuable information might be lost if the template is seen as a whole, since the semantics of the template cannot be represented. For example, some templates are similar in semantics but different in template indexes, using the statistical features or template indexes would ignore the semantic similarity and even induce false alarms [4].

The distributed representation methods apply text analysis to vectorize the log semantics. They look the log data as the natural text, each log message or log template is seen as a sentence composed of many words. By vectorizing the words of template, the template semantics is represented and the timing characteristics between continuous templates is extracted. Our approach falls into this distributed representation category. As they take the log template as a list of words, so the NLP techniques vectorize the words first, and then the template vectors are obtained based on word vectors. The commonly used NLP techniques include word2vec [39], fastText [40], Glove [41], Bert [42] and even the embedding layer based on neural network [31].

LogRobust [30] and research [43] respectively utilize fastText and Bert to vectorize words and then combine the word vectors through setting the TF–IDF of one timing window as the corresponding weights to represent log templates. LogOHC [44] and LogEvent2vec [35] utilize word2vec to embed each word into a k dimension vector first, and then average the word embeddings to represent log messages. Research [34] utilizes word2vec to embed the word into a k dimension vector first, and then constructs a 2D matrix whose size is $k * M$ to vectorize the template, where M is the word number of the longest template. nLSALog [31] uses word embedding layer based on neural network to train the log template number as input, and then uses the stacked LSTM with self-attention mechanism to detect the anomalies. Log2Vec [45] proposes a log-specific word embedding (LSWE) method which integrates lexical contrast into distributional vectors and strengthens semantics for determining similarity, and it also designs an out-of-vocabulary (OOV) word processor to assign a new embedding vector for the unseen words. Later, Template2vec [4] is proposed to extract the semantic information hidden in log templates, and then it aggregates all word vectors according to the TF–IDF to represent the template vector. LogTransfer [46] employs Glove to represent words. It extracts

the local context information and as well considers the global word co-occurrence information of the template set. After word embedding, the template vector is obtained by averaging all word vectors of each template.

Aggregating all word vectors not only can keep the dimension of the template vector fixed, but also can eliminate the influence of varying lengths of different templates. But the existing weighted aggregation methods taking the weighted sum of all words' vectors will discard the original order of the words [10]. Word order is important in semantics identification, discarding it might certainly result in semantics loss and ununiqueness of the representation [11,12,47]. For example, it cannot maintain the uniqueness between template vector and log template [35], which is exampled in the Section 3, and it might even influence the anomaly detection accuracy [13], which is proved in the experiment.

To maintain the word order in the semantics analysis, word-order preserving methods have also been explored. Research [12] optimizes the convolutional neural network model by embedding the word order characteristics in its convolution layer and polling layer to identify the deceptive opinions. Research [13] considers pairs of consecutive words to reduce the ambiguity generated by the word order loss, and then to perform the log mining based on NLP. They concatenate all the word vectors in order and make all template vectors' dimension same as the length of the longest template, which preserves the word order, but the obtained template vector's dimension is so big that it will increase the computation time cost [14].

In all, how to vectorize the semantics of raw log messages losslessly and balance the trade-off between the semantics representation and computation time is significant for improving further anomaly detection, especially how to combine the semantics of multiple logs to realize the real intelligent analysis. This paper puts emphasis on the efficient semantics representation of each type of log individually, and the semantics representation of multiple logs will be studied in the future work.

3. Problem statement

To utilize the valuable log data, many semantics representation methods have been explored to vectorize the raw log messages. The existing methods that aim to transform the raw log message into vector can be roughly divided into two categories. Weighted aggregation methods have been deployed in many scenarios [4,30,35,43,44,46,48] by aggregating all words' vectors based on their weights to perform anomaly detection tasks, and they are marked as "Weighted Average (WA)". It can be powered by some mature models to generate a word's vector, such as word2vec [39] and fast-text [40]. WA method can guarantee that messages of different lengths have the vectors of the same size, and the size of the converted vectors is smaller than that of concatenating all the words' vectors. But the original word order is lost and even the vector cannot represent the log message uniquely. To solve this issue, the word order-preserving methods concatenate all words' vectors [12,13] according to the original order they appear, and they have been deployed in [49], and it is marked as "Keep All (KA)". However, keeping all words might lead to a big size vector and then increase the computation time cost [14]. Therefore, how to design a semantics representation method that can losslessly and uniquely represent the log messages, and keep a relatively low computation time cost at the same time is the core problem to be solved in this paper.

Formally, a log message can be seen as a set of N words $M = \{d_1, \ldots, d_i, \ldots, d_N\}$, where a word d_i can be embedded into a vector whose length is k, $v_i = [v_{i1}, \ldots, v_{ik}]$. The message vector obtained by WA method is defined in Eq. (1). N is the word number of this log message, w_i is the ith word's weight in this message and it can mark its occurrence frequency or its importance of the word database. The length of each message vector is the same as that of a word, $1 * k$.

Table 1
Examples of word-to-vector representation.

Word order	Word	Weight (w_i)	Vector (v_i)
1	storage	0.2	[0.1, 0.2, 0.1]
2	interrupt	0.3	[0.2, 0.4, 0.4]
3	caused	0.1	[0.2, 0.4, 0.8]
4	input	0.1	[0.2, 0.2, 0.9]
5	errors	0.2	[0.4, 0.9, 0.2]
6	by	0.1	[0.3, 0.7, 0.9]

Table 2
Comparison of three vectorization methods.

Message vector	WA	KA	PE
Computation equation	Eq. (1)	Eq. (2)	Eq. (3)
Vector size	low($1 * k$)	high($N * k$)	mid($p * k$)
Unique representation	×	√	√
Consistent appear order	×	√	√

$$M_{v_WA} = \frac{1}{N} * \sum_{i=1}^{N} w_i * v_i \tag{1}$$

The message vector obtained from KA method is defined in Eq. (2). All the word vectors are concatenated according to the original order in log message. The length of each message vector is N times the word vector length, $N * k$.

$$M_{v_KA} = \underset{i=1}{\overset{k}{concat}}(v_i) = [v_{11}, v_{12}, \ldots, v_{ji}, \ldots, v_{Nk}],\ 1 < j < N \tag{2}$$

Take three simple study cases of BGL dataset [17] for example. Assume there are six words ($N = 6$) and every word is represented as a vector of k elements ($k = 3$). The corresponding weights and vectors of words are listed in Table 1.

Case 1: For two completely different messages "storage interrupt" and "caused input errors", their corresponding vectors obtained by Eq. (1) are the same [0.04, 0.08, 0.07], which proves the existence of semantics representation ambiguity issue in the WA representation method.

Case 2: For two messages "storage interrupt caused by input errors" and "input errors caused by storage interrupt" with different semantics, they have the same words but different orders, their corresponding vectors obtained by Eq. (1) are the same, which proves the existence of semantics loss issue caused by the WA representation method.

Case 3: For one message containing 6 words "storage interrupt caused by input errors", the lengths of vectors obtained by above two methods are different. The vector size obtained by Eq. (1) is the length of the word vector, 3, but the vector size obtained by Eq. (2) is the product of word number and word vector length, $3 * 6 = 18$. It is clear that the vector size of the latter is bigger than that of the former. This case proves the existence of the big size vector issue in the word order-preserving method (KA).

In all, as shown in Table 2, WA method is not good at lossless semantics representation, especially in terms of uniqueness and order consistency. On the contrary, KA method overcomes these drawbacks but its vector size is bigger, which might cause high computation time cost. Therefore, to overcome the above issues and balance these two methods, we propose a novel representation method which is called "Partial Extraction (PE)". It extracts theme words to retain the log template abstraction after discarding useless words, and then represents the log message through concatenating the extracted word vectors like Eq. (3), where p

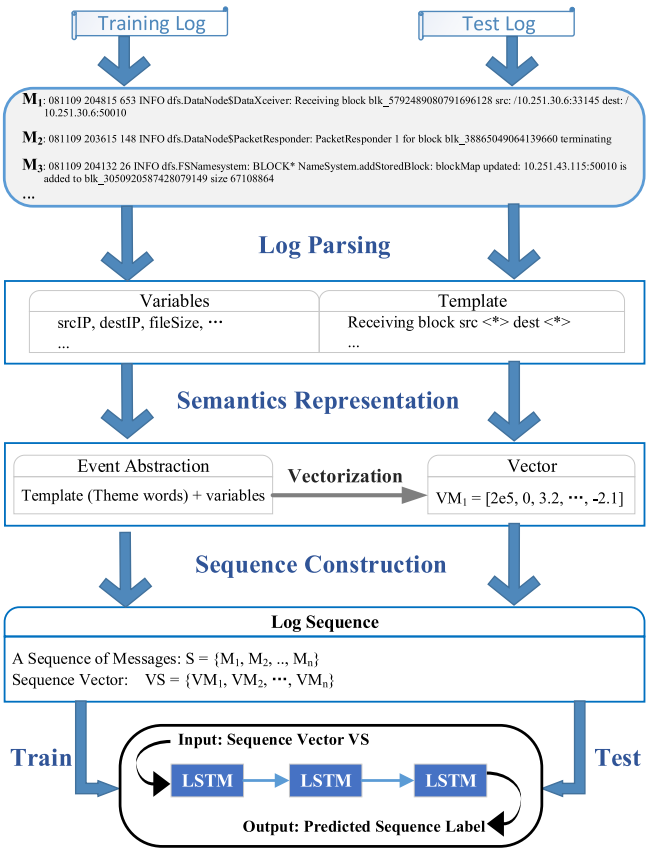


Fig. 2. Overview of LogNADS.

means the number of extracted theme words, and p value is determined empirically. The p word vectors are concatenated based on the order they appear in the log message, so that it can guarantee both the unique semantics representation and the original order relationship, and further to maintain near-lossless semantic knowledge.

$$M_{v_PE} = \underset{i=1}{\overset{p}{concat}}(v_i),\ 1 < p < k \tag{3}$$

4. LogNADS: A scheme to log-based network anomaly detection

To better figure out the system's runtime status, LogNADS is proposed in this paper, a novel anomaly detection scheme based on log semantics representation. The global view of LogNADS is shown in Fig. 2. The first step is log parsing on raw messages. Subsequently, LogNADS does not depend on the log count vector which has been used in the existing methods [9,50–54]. Instead, it extracts theme words from each log template to represent semantics, and then transforms these words into vectors. Owing to it, no matter what structure of the log event has, it can be represented as a vector with near-lossless semantics knowledge remained. Hence, the proposed representation method can also be able to handle heterogeneous logs collected from different systems. After semantics extraction and vector representation, the sequence data is constructed using the sliding window scheme and the designed optimal window size computation method. Finally, LSTM is built to detect sequential anomalies.

4.1. Log parsing

Raw log events are unstructured data, and contain specific information, such as IP address, responder ID and packet size.

Table 3
Parsing examples of HDFS dataset.

Raw events	(1) Receiving block *blk*_5792489080791696128 src: /10.251.30.6 : 33145 dest: /10.251.30.6 : 50010 (2) Received block *blk*_3587508140051953248 of size 67108864 from /10.251.42.84 (3) Got exception while serving *blk*_8466246428293623262 to /10.251.106.37
Templates	(1) Receiving block ⟨∗⟩ src: ⟨∗⟩ dest: ⟨∗⟩ (2) Received block ⟨∗⟩ of size ⟨∗⟩ from ⟨∗⟩ (3) Got exception while serving ⟨∗⟩ to ⟨∗⟩

Log parsing [32] aiming to transform the unstructured messages into the structured templates and variables is the first step for automatic log analysis [50].

Take the HDFS log dataset for instance, the transformation examples are shown in Table 3. LogNADS adopts FastLogSim [15] to pick out and replace the variables (specific information) from each raw event with ⟨∗⟩, and the remaining information is called template in this paper. The structured data after log parsing include two parts: template and variables. The latter part is numerical or easy to be changed into numerical, but it is worth noting that contents of the former part is still unstructured since it includes various numbers of words. So we design a semantics extraction method to represent the template abstraction in the next subsection.

4.2. Semantics representation

To represent the semantics of templates [55], a series of preprocessing operations are taken: removing noise words, splitting composite verb, de-duplicating templates and extracting dominant words.

(1) Removing all symbols from templates, such as delimiters, operators, punctuation marks, and then removing useless items, such as 'a', 'the', etc.

(2) Changing the capital letters into lower-case letters, and then splitting composite verb items. For example, 'addStoredBlock' will be split as 'add', 'stored' and 'block'.

(3) De-duplicating the same templates and merging similar templates to reduce the workload of semantics extraction.

(4) Extracting theme words. For example, a template "block ⟨∗⟩ is added to invalidSet of ⟨∗⟩" will be represented as theme semantics "added to invalid Set" which is similar with the original template, and can be seen as a near-lossless semantics representation.

After template de-duplication, millions of templates will be reduced to tens of distinct templates, and then the workload of semantics extraction will be relatively less. As suggested in the reported research [56,57], the words at the beginning of a log message are more likely to be part of log abstraction than words at the end of a log message. Thus, some useless words are first discarded automatically, such as the preposition (in, of, et al.) and article (a, an, the). And then four theme words are selected automatically from the beginning of the templates. Afterwards, LogNADS involves security operators to check the result, and fine-tune the extracted theme words for guaranteeing the lossless semantics, if needed. The former automatic extraction process can reduce the workload of security operators, and the latter human involvement can make the extraction more accurately. So we select this kind of human–computer interaction way in the semantics representation.

To balance the trade-off between semantics vector scale and computation time cost, and also guarantee the same size of different template vectors, we empirically control the length of the event fixed as 4 that is near the median of word numbers in

one template instead of the longest template, thereby helping to reduce the computation time cost. Generally, a complete sentence containing the subject, the predicate and the object can present the main meanings briefly, so we retain these three main parts and another specific value together. If the length of filtered template is less than 4, the empty location will be padded with the constant value.

4.3. Log vectorization

After the above two steps, an unstructured log message is represented as a sequence of words and variable values as shown in Fig. 3. LogNADS transforms each raw log message into a numerical vector to perform the downstream detection tasks. Some studies utilize template indexes or variable statistics to detect anomaly [29], which loses the semantic information. Among the variables, those variables easy to indicate the event label are discarded. For example, the block ID (*blk_ID*) of HDFS. Thus, we discard the *blk_ID* and some other variables to make the anomaly detection task more real. Then we keep the remaining source IP (srcIP) and destination IP (dstIP), and responder ID (resID) with the theme words extracted from template as the final semantic features of each message. Herein, the resID is originally numerical, and for IP addresses, we convert them into numerical digits. For one IP $a.b.c.d$, the converted digit is computed by $x = a * 256^3 + b * 256^2 + c * 256 + d$.

As suggested in the reported research [56,57], the words at the beginning of a log message are more likely to be part of event log abstraction than that at the end. We extract partial subject words and action words from each template, we find at most four words to give the best results, thus the number of theme words is set as four empirically. We first adopt the word2vec [39] to obtain the vector of each word. Normally, the weighted aggregation methods maintain the accurate template representation by a high-dimensional embedding word vector. On the contrary, we maintain the accurate and unique representation by the word order, so the words are embedded into a relatively low-dimensional vector. And then we concatenate the word vectors according to the order they appear in raw message. From the view of semantics integrity measurement, the vectorization must satisfy the following three requirements:

(1) The words of similar semantics must have a higher similarity distance that can be measured by Cosine, Euclidean and so on. For example, "receiving" and "receive" are semantically similar.

(2) The words of different semantics must have a lower similarity distance. For example, "success" and "failure" are semantically different.

(3) There must be the one-to-one relationship between log vector and log message. For example, a sequence $\{word1, word2, word3\}$ can be uniquely represented by $concat(vec1, vec2, vec3)$ instead of $\frac{1}{3} w_i \sum_{i=1}^{3} vec_i$, the example is shown in Section 3.

The introduction of word order not only helps to keep semantics but also makes the unique representation of different templates easier than using high-dimensional word vector, thus the words are encoded into a relatively low-dimensional vector to maintain the low computation time cost in this paper.

4.4. Sequence construction

To reach one objective, one user might take a sequence of actions, and sometimes to ensure the malicious objective furtive for the system security center, the user might take a sequence of normal actions to attain his goal. Take a simple case study as an example, the attacker sends a crafted e-mail to the victim, and the e-mail attaches an excel file with a malicious macro embedded. The victim reads the e-mail, and he/she might download excel file first, and then open it. At this time, the macro will execute a malware right after, and at last the malware will delete the blacklist of accounts [6]. All these sequential actions will be recorded in the log file chronologically. If judge one action independently, only the action "delete the blacklist of accounts" seems to be anomalous, while the others seem to be normal. Hence, how to detect the anomalous sequence is necessary.

Normally, a sequence S of actions can be represented by sorting the log messages by time, such as $S = \{[t_1, M_1], \ldots, [t_i, M_i], \ldots, [t_N, M_N]\}$ where t_i is time-stamp, normally represented as "YY/MM/DD HH:MM:SS", M_i is the ith message that can be transformed into a vector $V_i \in \mathbb{R}^k$ where k is the dimension of V_i, and N is the sequence length. With the help of the above vectorization method, one sequence vector is represented as $S = (V_1, \ldots, V_i \ldots, V_N)$ whose dimension is $N * k$.

To obtain the sequence data, windowing is applied to divide a log dataset into finite chunks [2]. The sliding window scheme is utilized to dynamically represent log sequences of different lengths. As shown in Fig. 4, the window size is the sequence length and the step size is the difference of two timely adjacent sequences. Since the real anomalous sequences usually have different lengths and they are even often accompanied by some normal actions, how to determine the optimal sliding window size and step size, influences the sequence anomalies detection. For example, if an attack includes 10 actions, but the sequence length is 5 in the detection scheme, which is difficult to capture the full trace of attack. On the other hand, if an attack includes 5 actions, but the sequence length is 20 in the detection scheme, which might include multi-attacks in one sequence and then confuse the detection scheme. Therefore, how to determine the optimal window size and step size is considered in this paper.

We design a method to compute the window size according to the information gain and the data distribution during a period time. The computation method is shown in Eq. (4), where N_{maj} and N_{min} indicate the sample number of majority class and minority class. Normally, to maintain the completeness of the sequence, the step size is less than the window size. Hence, to balance the sequence length and sequence completeness, we set the sliding size of two adjacent windows as 75% of the window size in order to capture the case in which one or more anomalous events appear in two subsequent sequences.

$$wind_size = \lceil [\frac{N_{maj} * logN_{maj} + N_{min} * logN_{min}}{2 * (N_{maj} + N_{min})} + \frac{N_{maj}}{N_{min}}]/2 \rceil \quad (4)$$

$$step_size = \lceil wind_size * 75\% \rceil \quad (5)$$

Except for the sliding window scheme, we also consider the session scheme in this paper, which is realized by session window through grouping messages based on identifiers instead of sorting by time-stamp directly. For example, HDFS log can also be grouped by blk_{id} to obtain the session log data which record the operations of each block.

This paper focuses on designing a sequence anomaly detection scheme which can automatically vectorize a sequence of events, and then using these sequence vectors to train a supervised classification model to detect the new sequential events. To build the training sequence data and label, if a sequence contains at least one anomalous action, this sequence will be labeled as anomalous. If a sequence is detected as anomalous, the specific cause of the anomalous sequence will be investigated by the domain expert deeply that is not studied in this paper.

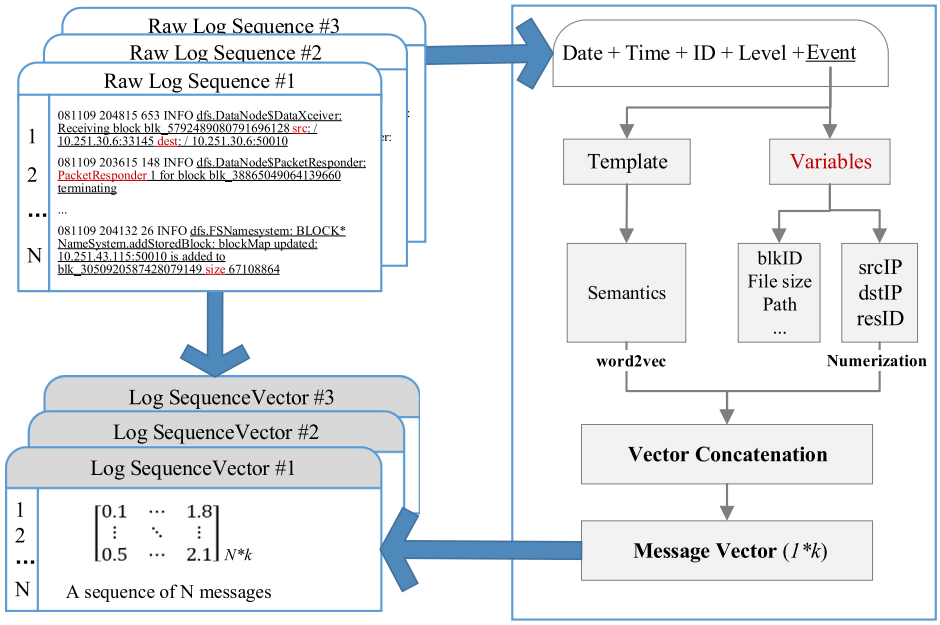


Fig. 3. Flow of Vectorization.

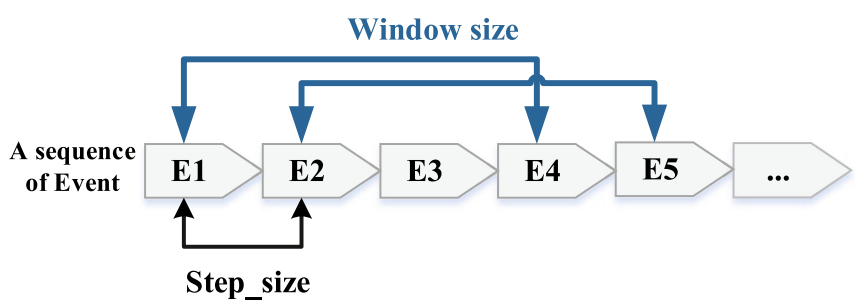


Fig. 4. Log sequence extraction based on sliding window.

4.5. Classification model

To detect the sequential anomalies, the detection task is mapped into a supervised binary class classification on the vectorized sequence data.

In many deep learning models, recurrent neural network (RNN) is popular for its network structure containing the characteristics of learning timing feature, which makes it more adaptive in the analysis of timely sequential data. So RNN provides solution for our purpose. Among the variants of RNN, the Long Short-Term Memory (LSTM) [27] model compensates for the weakness of gradient disappearance, gradient explosion, and the lack of long-term memory ability, so that the RNN can make full use of the long-distance time sequence information. Meanwhile, compared with the early RNN model, the cell of LSTM has been improved by introducing three gates which can discard the useless information transformed from the last cell and then retain the useful information going to transform to the next cell. Generally, every action of timing sequence can be input into one cell to process, and the knowledge of our sequence data can also be transformed by this measure. Transforming only useful information between cells can lighten the model's learning workload. Hence, it seems possible and promising to apply LSTM in this paper to perform a binary classification task to achieve the network anomaly detection.

The overview of the detection model is illustrated in Fig. 5. The unit of LSTM is the cell, and there is a cell state C_i which passes throughout all cells, which can be used to store the old information, x_i is the current state and h_i is the hidden state and also the output state. For each cell of LSTM, the input includes two parts: the current state x_i and the hidden state h_{i-1} that is transmitted from the last cell. In each cell, there are three gates to handle the input states, namely forget gate, input gate and output gate. Forget gate is the first layer to discard some useless information, input gate is the second layer to reserve some information, and output gate is the third layer to transmit useful information to the next cell. The difference between the three gates is the weight and bias of the layers' activation functions. Finally, three gates' outputs are combined with the current cell's input state x_i to obtain the final hidden state h_i and then transmit it to the next cell. Finally, only the last cell's output state h_n is fed to the next Dense layer.

In Fig. 5, the input of LSTM is a batch of timing sequence data. A sequence data is marked as $S = \{x_1, \ldots, x_i, \ldots, x_n\}$, x_i is ith log message's vector. After the learning process, the hidden state h_n generated from the last cell is transformed into the next layer. The activation of Dense layer is *tanh*. Finally, the predicted label y_pred for this sequence is computed by loss function *softmax*, and it is defined as True or False to indicate whether this input data is anomalous or not. Since our focus is to measure the effectiveness of semantic representation method on anomaly detection, this paper just judges whether the sequence is anomalous or not, and does not reveal the specific reason causing anomaly.

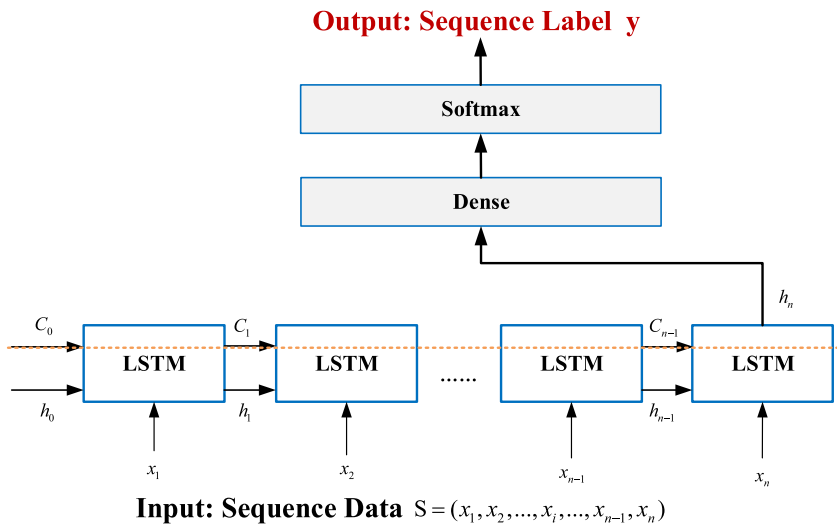


Fig. 5. Structure of LSTM.

5. Experiments

We present five cases to evaluate the proposed approach LogNADS. The objectives of experiments are listed as follows:

(a) Comparing with state-of-the-art methods.
(b) Comparing different semantics representation methods.
(c) Evaluating different lengths of sequences.
(d) Evaluating different lengths of events.
(e) Analyzing the statistical significance test.

5.1. Experiment configuration

In the real system, after collecting the relevant logs. The history raw logs will be parsed into templates first, and then the template vectors are obtained by the proposed semantics representation method of LogNADS and then the template vectors are stored in a library. All the log templates will be represented as vectors after the preprocessing, and the proposed detection model will be trained on them. When the new log generates, it will be parsed first and then indexed into the existing template vector library to match a vector, and then a slicing window of logs will be input into the detection model to test whether this sequence is anomalous or not. To compare with other approaches, and to demonstrate the effectiveness of the LogNADS, we select two open source datasets that are commonly used in previously published papers and well known in the research domain. To avoid bias, each experiment has been run 20 times and the average results are reported. In the process of detection model training, we use the predicted label and ground-truth provided in the dataset to calculate the loss according to the mean squared error, set the learning rate as 1e−3, and use adaptive moment estimation optimization algorithm (Adam) [58] to tune the parameters of LSTM.

5.1.1. Dataset

We evaluate the proposed LogNADS on two public benchmark datasets: supercomputer log BGL [17] and the distributed system log HDFS [16]. HDFS is presented into two types, session-related and message-related. The datasets details are listed in Table 4.

(1) HDFS dataset

It is a commonly-used public benchmark dataset for log-based anomaly detection [16]. It contains 11,175,629 log messages generated from more than 200 Amazon's EC2 nodes, where 288,250 messages are anomalous that account for less than 2.6%. It can also be counted by session window scheme according to *block_id*, and there are 575,061 sessions of the dataset, among which 16,838 are anomalies that account for less than 2.9%. The anomalies are labeled by Hadoop domain experts [29].

Table 4
Details of the used datasets.

Datasets	BGL	HDFS	sHDFS
normal messages	4,399,503	10,887,379	254,979
anomaly messages	348,460	288,250	179,612
normal sessions	–	558,223	10,000
anomaly sessions	–	16,838	10,000

To validate the effectiveness of the proposed LogNADS in anomalous sequence detection, we randomly select 10,000 normal sessions and 10,000 anomalous sessions of HDFS dataset according to their session ID *block_id*, and then synthesize a session dataset (called sHDFS).

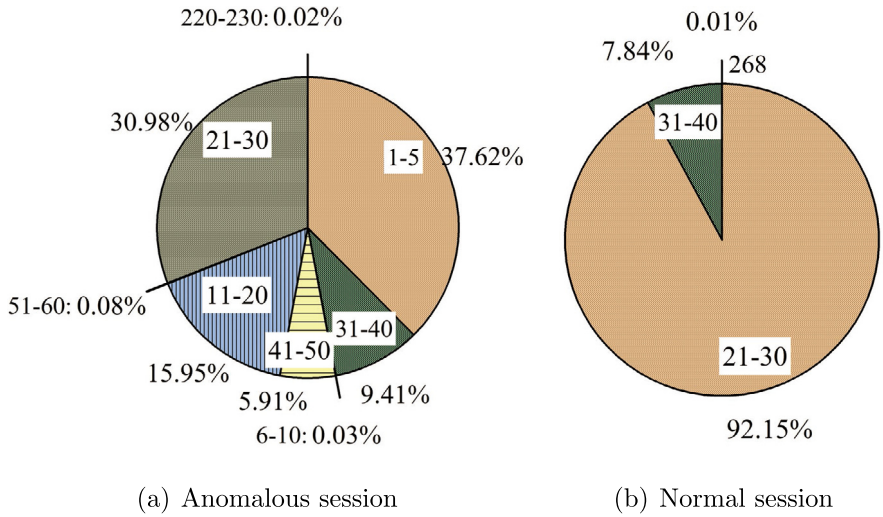
(2) BGL dataset

BGL dataset contains 4,747,963 log messages collected from the BlueGene/L supercomputer system at Lawrence Livermore National Labs (LLNL) [17]. Different from HDFS dataset, it has no identifier for each session. Thus, we use sliding windows to slice log messages into sequences according to the sliding window size and step size in Eq. (4) and Eq. (5). 348,460 log messages are labeled as anomalies. Here, a log sequence will be labeled as anomaly if any anomalous message exists in the sequence.

In contrast, HDFS dataset includes the session data of different lengths, and the length distribution is presented in Fig. 6. It can be seen from Fig. 6(a) that the anomalous session lengths fallen in $[1, 5]$ and $[21, 30]$ account for the most, followed by $[11, 20]$ and $[31, 40]$. In Fig. 6(b), the normal session lengths fallen in the range of $[21, 30]$ account for the most, followed by $[31, 40]$, and only less than ten sequences have the length of 268. According to the above statistical results, the length of anomalous sequence is relatively shorter than that of normal sequences.

5.1.2. Evaluation metric

Anomaly detection is abstracted as a binary classification problem in this paper. To validate the effectiveness, the commonly used metrics are selected. $Accuracy = \frac{TP+TN}{TP+TN+FN+FP}$, $Precision = \frac{TP}{TP+FP}$, $Recall = \frac{TP}{TP+FN}$, $F1 = \frac{2 \cdot Precision \cdot Recall}{Precision+Recall}$, $FPR = \frac{FP}{FP+TN}$. Herein, TP means the number of anomalous sequences classified correctly, FP means the number of normal sequences



(a) Anomalous session (b) Normal session

Fig. 6. Session length distribution of HDFS dataset.

incorrectly classified [59]. In addition, ROC (Receiving Operation Curve) and AUC (Area Under Curve) maximization has been used in state-of-the-art works to seek an effective classifier [60], so they are also used in this paper.

$$AUC = \frac{Recall + (1 - FPR)}{2} \tag{6}$$

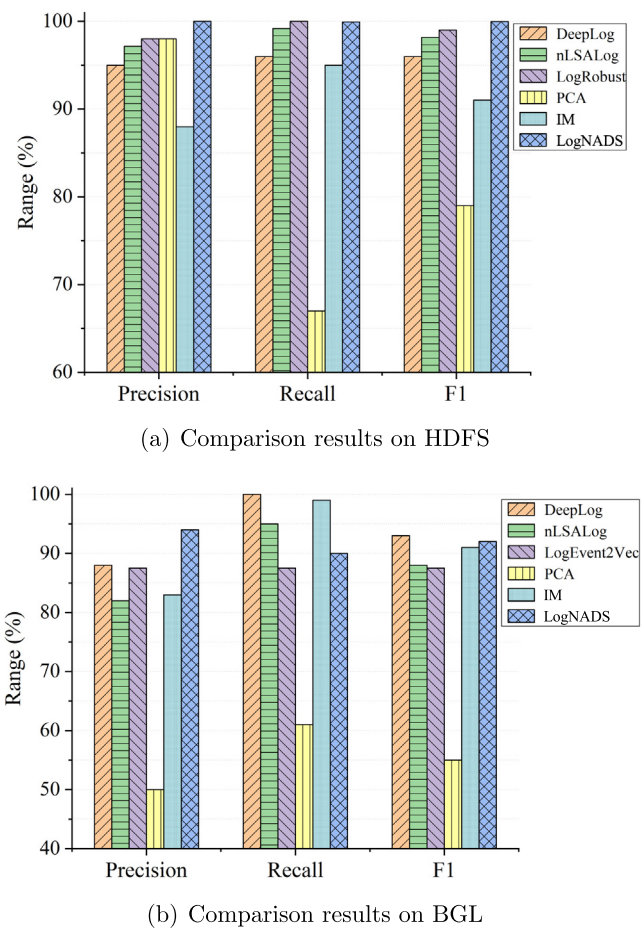
5.2. Comparison of state-of-the-art methods

This section describes the comparative detection results analysis of the proposed LogNADS and the reported results of other approaches, and further compares the results of different machine learning algorithms trained on the training data that are represented by the proposed semantics representation method.

5.2.1. Comparison with reported methods

We select three anomaly detection methods that did not consider the semantics to prove the necessity of semantics representation, and we also select another three anomaly detection methods that considered semantics based on weighted average approach to prove the superior performance of LogNADS. These six methods are used as the baselines to be compared with ours, and the detection accuracy are evaluated from three perspectives of Precision, Recall, and F1. Fig. 7 shows the comparison results of LogNADS and other state-of-the-art methods on HDFS dataset and BGL dataset, respectively. The state-of-the-art methods used to compare with ours are discussed here.

DeepLog [29] focused on using log template indexes and parameter values as input instead of semantics, and then using LSTM to predict a sequence's next event. If the top n events predicted for a sequence do not include the ground truth, it will be flagged as anomaly. **LogRobust** [30] focused on using weighted aggregation method to extract the semantic information of log event, and then using the attention-based Bi-LSTM to find the unstable log events and sequences. **LogEvent2Vec** [35] focused on extracting the relevance among log events by taking the template as the basic operating unit to train the word2vec, and then obtaining the same-size vector of templates to perform sequential anomalies detection based on Random Forest, Naive Bayes and Neural Network. **PCA** [8] focused on using log template indexes as input instead of semantics, and then using PCA to construct normal and abnormal space of log template count vectors. The log template count vectors far from the normal space are detected as anomalies. **Invariant Mining** (IM) [9] focused on using log template indexes as input instead of semantics, and then mining the invariants among log events from log template count vectors. The log sequences violating the invariants are detected as anomalies. **nLSALog** [31] used word embedding layer based on neural network to train the log template number as input, and then used the stacked LSTM with self-attention mechanism to detect the anomalies.



(a) Comparison results on HDFS

(b) Comparison results on BGL

Fig. 7. Comparison results with state-of-the-art methods.

It can be seen from Fig. 7 that LogNADS achieves the overall best metric values. The recall, precision, and F1 values of LogNADS are close to 1 respectively on sHDFS, which are all the best. The standard deviations of 20 groups of the precision, recall and F1 values are 0.00008, 0.00037 and 0.00018, respectively, which illustrates the stable results. The average accuracy value on sHDFS is also close to 1, and the standard deviation is 0.00014. On the BGL, all the evaluation values of LogNADS are more than 0.9, and the precision and F1 are nearly the best. The standard deviations of 20 groups of the precision, recall and F1 values are 0.12, 0.07 and 0.06, respectively, which is also promising. The average accuracy value on BGL is about 0.98, and the standard deviation is 0.01.

Two unsupervised methods of PCA and IM have unstable accuracy with low recall value and low F1 value on sHDFS and low precision value and low F1 value on BGL. Especially for the IM, its overall performance is imbalanced though its recall value is relatively bigger. On the contrary, for three supervised methods, LogRobust, DeepLog and nLSALog obtain relatively better results. DeepLog's recall value and F1 value on the BGL dataset are the best, but the precision value is less than that of the LogNADS, which can be reasoned by the semantics loss of DeepLog. LogRobust and nLSALog both perform a promising result on sHDFS, and the metric values are slightly less than that of LogNADS, since it considers the semantics in the process of representation. In consequence, the supervised methods perform globally better than that of the unsupervised methods.

PCA, IM and DeepLog leverage the log template index to learn the normal and abnormal patterns. Ignoring the common semantic information, both PCA and IM generate more false alarms than LogNADS. DeepLog combines the parameter statistics value vector to detect anomalies, its detection result is finally obtained according to the log template anomaly detection and parameter value anomaly detection. Hence, it performs better the PCA and IM. LogEvent2Vec regards the template as the independent unit, which captures the timing characteristics of sequences but ignores the semantics among words of the template. nLSALog, LogRobust and LogNADS employ semantics vectorization method to learn the semantic information of log templates, and effectively avoids high false alarms owing to that the semantics vectorization compares the semantics similarity of templates, while PCA, IM and DeepLog compare the similarity of template index instead of semantics. Especially, the proposed LogNADS presents a more balanced result than nLSALog and LogRobust because it keeps the original word order of templates while the other weighted aggregation methods fail to keep the word order.

5.2.2. Comparison of detection algorithms

To test the robustness and adaptability of the proposed semantics representation strategy, we implement it on different traditional machine learning methods. Random Forest (RF), Logic Regression (LR), Decision Tree (DT), K-Nearest Neighbor (KNN), and XGBclassifier (XGB) are used as the baseline detection algorithms. Fig. 8 exhibits the comparison results on sHDFS and BGL. Since LSTM is good at learning the timing characteristics, and the log sequence obeys some type of temporality, so it performs better than other detection algorithms. Moreover, most of them demonstrate better results, which suggests the general adaptability of the proposed semantics extraction and vectorization method on different machine learning algorithms. RF obtains a relatively better result than other machine learning algorithms because it is a set of decision trees and ensembles the learning ability of multiple trees.

In all, the proposed semantics representation method is promising for anomaly detection, and it prompts the LSTM to better learn complex semantic knowledge and timing characteristics compared with state-of-the-art detection algorithms.

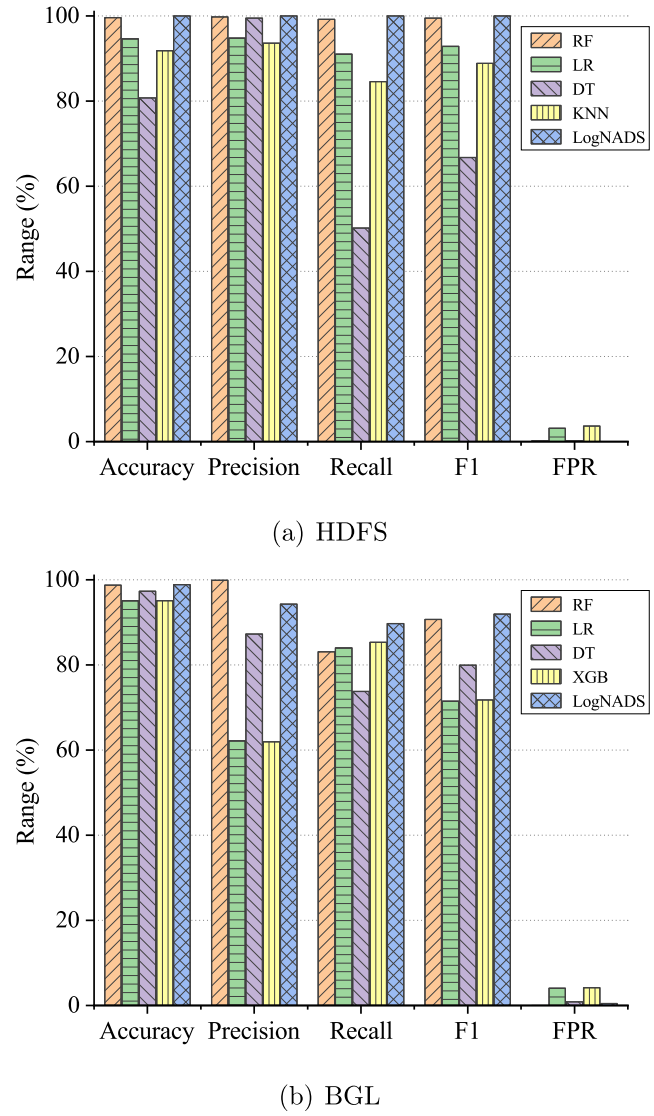


Fig. 8. Results of different detection algorithms.

5.3. Comparison of semantics representation methods

Since the objective of semantics representation is to perform anomaly detection, we implement the same detection model, LSTM on vectorized data obtained from different representation methods to compare the effectiveness of semantics representation. We deploy three semantics representation methods on the LSTM, and analyze the comparison results from the perspectives of detection accuracy and time performance.

The weighted average method is one commonly used method, and it is short for "Weighted Average (WA)" in this subsection. It aggregates all word vectors but fails to keep the word order, which has been proved to occur semantics loss [11,12]. The dimension of the converted template vector is equal to the word vector's dimension, and it is fixed and the smallest of three semantics representation methods. It has been deployed in approaches [4,30,55].

Word order-preserving method retains all words according to their original appearing order, and it is short for "Keep All (KA)". It concatenates all word vectors with the original word order, and it can be seen as no semantics loss but it is not compact theme. The dimension of converted vector is unfixed and normally the

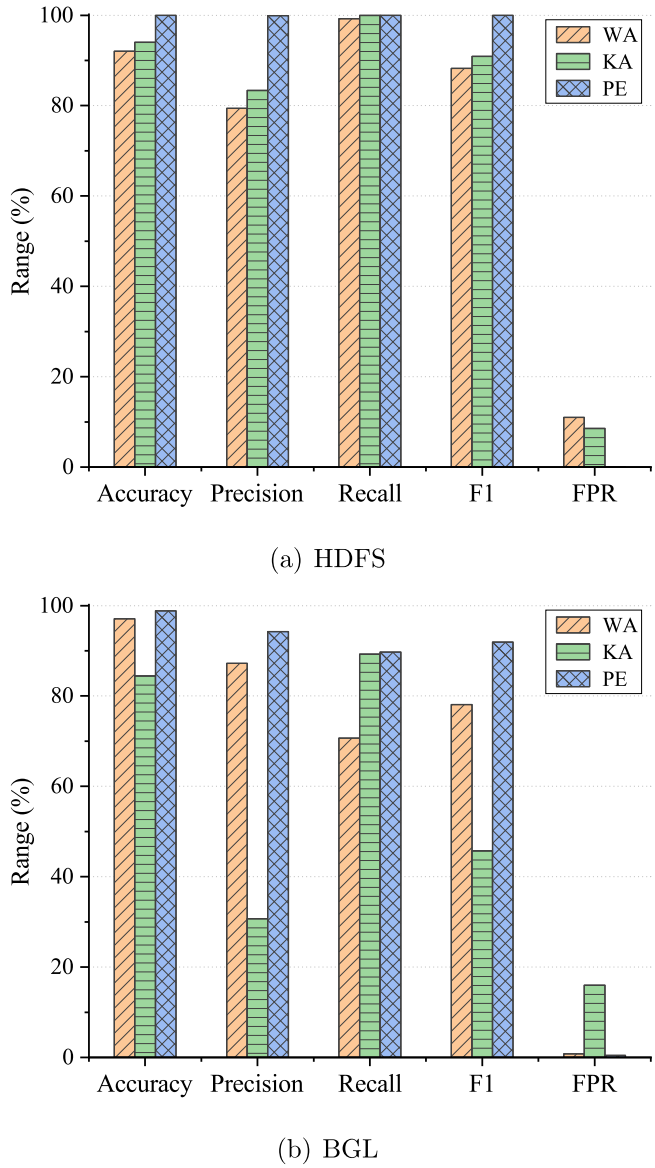


Fig. 9. Results of different semantics representation methods.

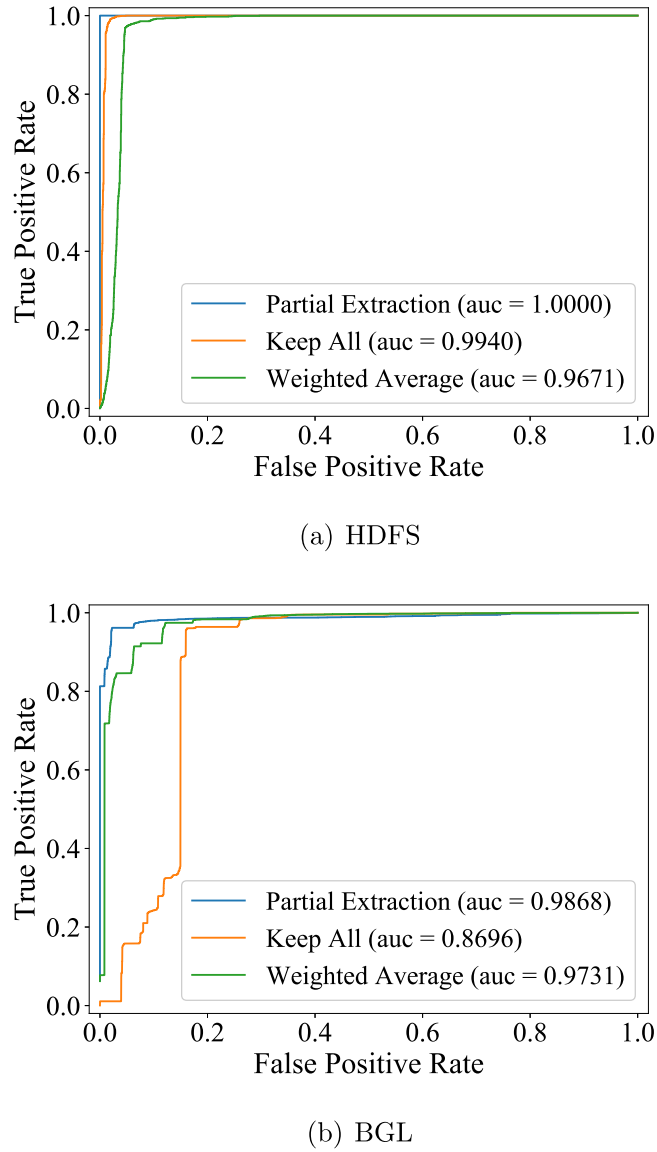


Fig. 10. ROC curves of different semantics representation methods.

biggest dimension of three semantics representation methods, which has been proved in the example in Section 3. Generally, the dimension of converted template vector is equal to the word vector's dimension times the word number in the longest template. It has been implemented in the approaches [12,13,49].

"Partial Extraction (PE)" proposed in this paper can be seen as a balanced solution, it keeps the theme words and their order to retain near-lossless semantics. The dimension of converted vector is controlled by the template length, which is the number of theme words. The dimension of converted template vector is equal to the word vector's dimension times the theme words number in each template.

5.3.1. Anomaly detection accuracy

The detection accuracy results and Receiver Operating Characteristic (ROC) curves of three semantics representation methods on sHDFS and BGL datasets are exhibited. Comparing the results of Figs. 9 and 10 on the same dataset, it can be found that the proposed "Partial Extraction" method shows a better performance than the other representation methods, and the superior performance is obvious on both datasets. All the evaluation metric values are the best where the true detection rates are high and false rates are low, as well as a high AUC in the ROC curve. Though "Keep All" method keeps all words and their order to contain lossless semantics knowledge, concatenating all words will generate high-dimensional data, and further confuse the detection model to learn the theme features. For the "Weighted Average" method, it performs better than "Keep All" since it aggregates all word vectors by weight to avoid the high-dimensional and complex data of "Keep All" method, but it is slightly worse than "Partial Extraction" due to the word order loss exposed by it. Consequently, the proposed "Partial Extraction" method achieves the best results. It achieves a trade-off through avoiding the semantics loss and word order loss exposed in the other methods.

5.3.2. Time performance

Three representation methods generate different vector scales, we further compare the test time on one sequence of sHDFS and BGL. According to the 20 groups of test results, the average time and the standard deviations are exploited in Table 5, and they are separated by the symbol '/'. The unit of test time on sHDFS and BGL is millisecond and microsecond. The best results are marked

Table 5
Time comparison of three vectorization methods.

Test time	Partial extraction	Keep all	Weighted average
HDFS(ms)	0.034/0.06	0.038/0.062	**0.033**/**0.058**
BGL(us)	22.91/**0.46**	43.64/0.47	**15.36**/0.56
Training time	Partial extraction	Keep all	Weighted average
HDFS(ms)	1.225/**0.309**	1.235/0.432	**1.218**/0.335
BGL(us)	161.14/**1.1**	497.97/1.33	**116.45**/1.46

in bold. Normally, the scale of vectors generated from "Keep All" is the largest, followed by "Partial Extraction", and "Weighted Average". Observations can be found: (i) Big scale vector is easy to cause high computation time cost, which is expressed as a large train or test time. (ii) Comparing test time difference, "Keep All" method's test time is obviously bigger than the others, and the "Partial Extraction" method's time is slightly more than the best value of "Weighted Average" method. Hence, "Partial Extraction" approximately achieves the best accuracy results with a lower test time.

For the test time, the standard deviations of "Weighted Average" on HDFS is the least and that of "Partial Extraction" on BGL is the least, which illustrates the stable test time of these two methods. On the HDFS, the standard deviation of "Partial Extraction" is 0.06 that is a little bigger than that of the best value, which is also relatively stable. For the training time, both standard deviations of "Partial Extraction" on HDFS and BGL are the least, which illustrates the stable training time of "Partial Extraction".

Overall, judging from detection accuracy and time performance results, our representation method based on semantics representation has achieved high detection accuracy and low computation time cost. Furthermore, LogNADS has a superior performance and it outperforms other state-of-the-art semantics representation methods. Therefore, the semantics extraction method proposed in this paper might be an alternative method for the fast log-based anomaly detection in the future.

5.4. Evaluation of different sequence lengths

AUC is often used to evaluate the generalization ability of the detection model. ROC and AUC maximization has been used to seek an effective classifier in the existing research [60]. We use them to validate the effectiveness of the proposed sequence construction method of LogNADS, and then evaluate the influence of sequence length on the sequence anomaly detection accuracy. We construct four datasets by varying the sequence length, where the sequence length indicates the number of events in one sequence. Since sHDFS dataset is session-based, and the label is based on the session, so we test it on the BGL dataset. Four groups of detection results from BGL dataset are elaborated.

5.4.1. ROC analysis

Normally, the sequence length varies with the environment configuration. To validate the optimality of the computed sequence length of Eq. (4), and as well as to test the adaptability of LogNADS on different sequence lengths, we control it by adjusting the sliding window size. In BGL dataset, the computed sequence length is 10, and we further vary it in the set of {10, 20, 30, 40} on BGL. Fig. 11 plots the ROC curves of different sequence lengths. It can be found that, when the sequence length is more than 20, the more sequence length is, the more AUC is. Since some sequences might correspond exactly to some sub-sequences of the normal sequences that are used to train the classification model. However, if some anomalous sequences have few steps, the short sequence might be more suitable. For example, in BGL, the short sequence is more suitable for detection sequential anomalies. The sequence length of BGL dataset is set as 10 without description in this paper where the AUC is the best when the sequence length is 10.

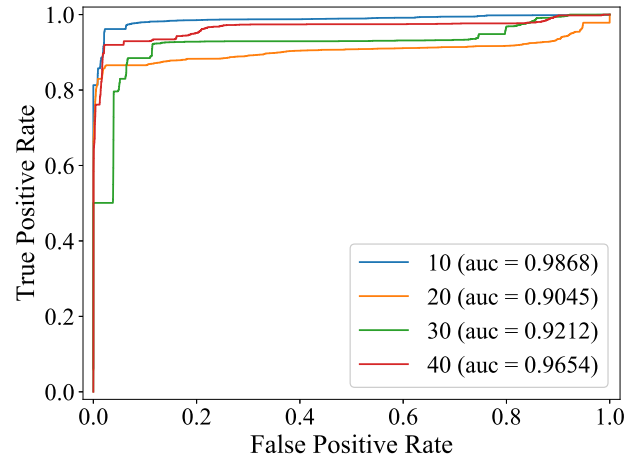


Fig. 11. ROC curves of different sequence lengths on BGL dataset.

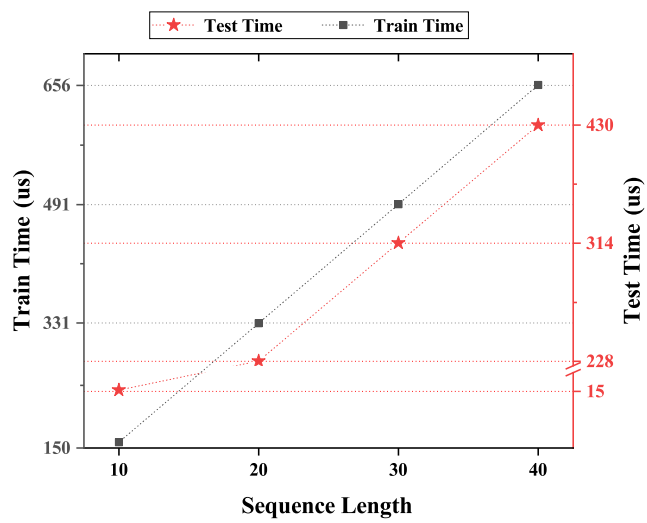


Fig. 12. Time values of different sequence lengths on BGL dataset.

5.4.2. Time performance analysis

The time performance results on BGL are presented in Fig. 12. It contains training time on the left Y-axis and test time on the right Y-axis, and the unit is microsecond (us). It can be found that, both time results increase with the increasing sequence length. The test time of one sequence containing 10 events is less than 1 millisecond, which is promising and possible to be improved for timely-detection in the future.

In all, considering both detection accuracy and time performance, we set the sequence length as 10 to balance the trade-off between detection time and detection accuracy. The optimal sequence length computation method is proved effective on BGL dataset, and it can also be used in other big dataset to automatically construct sequence data for improving the detection accuracy.

5.5. Evaluation of different event lengths

We construct eight datasets by varying the event length, where the event length indicates the number of the extracted

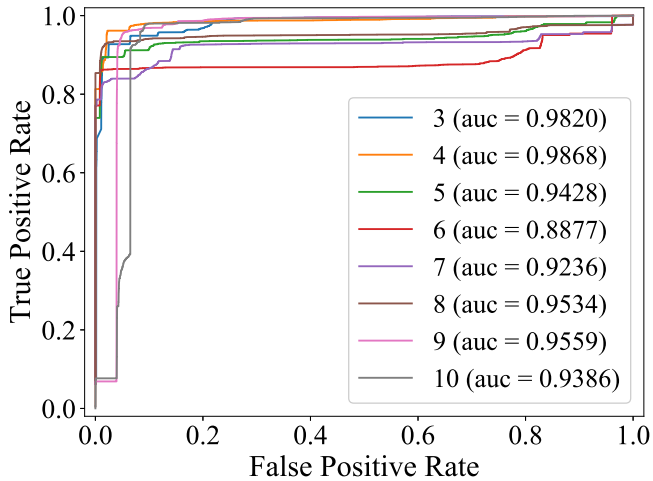


Fig. 13. ROC curves of different event lengths on BGL dataset.

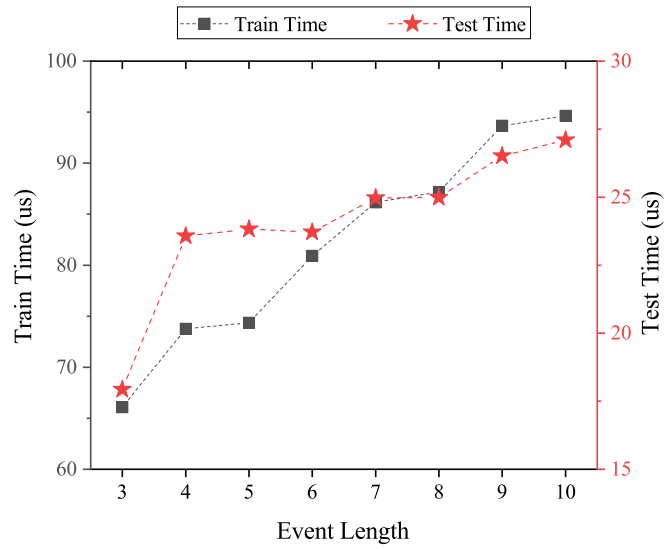


Fig. 14. Time values of different event lengths on BGL dataset.

theme words in each log template. Eight groups of detection results obtained from BGL dataset are elaborated.

5.5.1. ROC analysis

The event length is the number of the extracted theme words of one log template in this paper. Different sources of log data usually have different log templates. Generally, the event length varies with the environment configuration. To test the influence of event length on the anomaly detection performance and to determine the optimal value of length, we control it by adjusting the event length from 3 to 10 on the BGL.

Fig. 13 shows the ROC curves of different event lengths on BGL. Normally, the more AUC is, the more stable and better the detection algorithm performs. It can be found that there is no robust relation between the event length and AUC. But, it reflects a phenomenon that a less event length is more likely to contribute a higher AUC. In this paper, the event length is set as 4 where the largest AUC is obtained. On the one hand, we extract 4 theme words which include the subject, predicate, object and another component of one log template, which are able to represent the main meaning, and they are also easy to differentiate with others. On the other hand, the fewer words extracted, the less human resources are needed to check them. In contrast, a long log template might contain some common words with other log templates, which will confuse the classifier. Consequently, maintaining all words will not ensure a better detection result necessarily, and determining an appropriate length of template is thus necessary.

5.5.2. Time performance analysis

The corresponding time results on BGL are presented in Fig. 14. It contains training time on the left Y-axis and test time on the right Y-axis. Both time results are measured in microsecond (us). It can be found that, with the increasing of event length, the time increases accordingly. Since the training time not only depends on the volume of training dataset but also depends on the optimizer and the other configurations, the increasing trend of training time is not smooth as that of the test time.

Therefore, considering the detection accuracy and the time performance, we set the event length as 4 to balance the trade-off between detection time and detection accuracy.

Table 6
Significance test results along with Friedman rank.

Vectorization	Classifiers	AUC		ACC		F1	
		Rank	p-value	Rank	p-value	Rank	p-value
	LSTM	**1**		**1**		**1**	
PE	RF	2		2		2	
	DT	5.67		4.67		5.33	
	LSTM	5.33		5		5	
WA	RF	5	0.0446	4.67	0.0498	5	0.0319
	DT	8.67		8.33		8.67	
	LSTM	5.33		7		6	
KA	RF	4.33		4.33		4.33	
	DT	7.67		8		7.67	

5.6. Significance test analysis

To strengthen our approach, the statistical significance tests are conducted to compare the performances of various approaches on multiple datasets. Friedman test [61] and post-hoc Nemenyi test [61] are used to further analyze whether our approach is statistically significant compared with others. As shown in Table 6, the AUC, ACC and F1 values of nine methods over the HDFS and BGL datasets are demonstrated. Nine methods are developed by combining three vectorization methods and three classifiers, herein the combination of "PE" vectorization method and LSTM classifier indicates LogNADS. After Friedman hypothesis testing, the null hypothesis (the performances of all approaches are equivalent) is rejected at $\alpha = 0.05$ since the p-values of AUC, ACC and F1 metrics are all less than 0.05. This result indicates that there is at least one method that has performance difference among the competitors, respectively.

Afterwards, it needs to conduct the post-hoc test to further measure how significant are the performance differences among the considered approaches. The post-hoc Nemenyi test is adopted to identify the pairs that indicate statistical differences between different methods. The critical difference (CD) of 6.9363 is computed at $p\text{-}value = 0.05$. In Table 6, the best value is indicated in bold. LogNADS appears as the best of the benchmark approaches, so it is picked as a control algorithm for being compared with the remaining approaches in the post-hoc test. It can be found that the methods including DT perform more significantly different than other classifiers, and the methods including "WA" also perform more significantly different than other vectorization

methods. In summary, LogNADS performs relatively better than other methods in the metric values and significance test.

6. Threats to validity

In this section, we discuss possible validity threats [62] regarding for our experimentation.

Threats to **internal validity** are concerned with factors that may affect the results. The internal validity of the method could be threatened by the choice of the event length value and the word vector length value. These two values are not fixed and might lead to different accuracy results. To mitigate any bias in this respect, we tried many values of different event lengths and the word vector lengths respectively while kept the other configuration unchanged, and finally selected the optimal parameter values.

Threats to **external validity** are concerned with the generalizability of the results. A single labeled benchmark dataset might be one of the possible threats. We mitigated this threat by selecting two representative datasets that are commonly used in the relevant papers and collected from two different scenarios, and we saw that the proposed method is applicable to the datasets of these two types. Another external validity threat is the new identified log template, LogNADS does not consider the OOV (Out-of-Vocabulary) problem, so the unseen template cannot be indexed in the existing template vector library, this requires manual processing to match the new pattern into a similar pattern vector for next analysis task. It will be studied in the future work. If this problem can be solved, the online detection will be realized as well.

Threats to **conclusion validity** are concerned with the correctness of the obtained results. The threat to the conclusion validity exists in the use of only one anomaly detection technique, LSTM. It is possible that other detection techniques might outperform LSTM on the same represented sequence data. We mitigated this threat by the fact that LSTM is good at learning timing characteristics of sequence data, and we also introduced other four popular machine learning techniques to be compared with LSTM on the same data, and the comparison results proved the validity of LSTM in this paper.

We believe that these threats, while worth noting, do not invalidate our results or diminish their value. We do not claim that our methodology is universal or applicable to all the scenarios. Instead, with this study we explore an alternative application of log semantics representation in network anomaly detection, and demonstrate that these approaches help to provide actionable insights to security operators.

7. Conclusion and future work

Logs have been one of the most precious data sources for network security maintenance, but how to utilize the semantics of logs to improve the sequence anomaly detection is especially challenging. Thus, the anomaly detection schemes based on log semantics are needed.

We have proposed a log semantics-based scheme, LogNADS for detecting the sequential anomalies. Our main objective has been to extract the log semantics and then to learn the temporal dependencies knowledge between log sequences, and thus identify the sequential anomalies precisely. To achieve this, we have designed a novel near-lossless semantics representation method. Each log template is regarded as a sentence consisting of many words. The useless words are discarded, and the extracted useful ones' vectors are concatenated according to their order, and then the log template vector is constructed. To detect the sequential anomalies, we design a sliding window size computation method to automatically construct the optimal length of sequence data for the downstream detection task. LSTM is used to extract the complex temporal dependence of sequence data and further to detect sequential anomalies. Contributed by these steps, LogNADS not only overcomes the existing challenges on semantics loss but also realizes an automatic sequence data construction for improving network anomaly detection. Extensive experiments on the HDFS and BGL datasets demonstrate the effectiveness and superior performance of LogNADS to state-of-the-art methods, and also confirm the optimal parameter configuration including sequence length, event length and detection algorithm. In all, our research has demonstrated that LogNADS can effectively detect the sequential anomalies with the help of log semantics.

In this paper, we have proposed a sequence detection scheme involving log semantics representation and automatic sequence construction. The emphasis of semantics representation is on a single type of log individually. In the real system, the security expert usually analyzes the security status based on multiple types of logs together. Therefore, the semantics representation of multiple logs will be studied in the future work. In addition, LogNADS is not updated incrementally, the online incremental training will be considered in the future work to solve some sudden attacks might be occurred and also to further improve the analysis efficiency of big data in the real system.

CRediT authorship contribution statement

Xu Liu: Conceptualization, Methodology, Formal analysis, Validation, Writing - original draft, Writing - review & editing. **Weiyou Liu:** Investigation, Validation, Visualization. **Xiaoqiang Di:** Supervision, Project administration, Writing - review & editing, Funding acquisition. **Jinqing Li:** Visualization. **Binbin Cai:** Resources. **Weiwu Ren:** Software, Funding acquisition. **Huamin Yang:** Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the Science and Technology Development Plan Project, Jilin, China, under Grant 20190302070GX, and in part by the Education Department of Jilin Province under Grant JJKH20200795KJ, Grant JJKH20200800KJ and Grant GH180148.

References

[1] M. Landauer, F. Skopik, M. Wurzenberger, A. Rauber, System log clustering approaches for cyber security applications: A survey, Comput. Secur. 92 (2020) 101739, http://dx.doi.org/10.1016/j.cose.2020.101739.

[2] S. He, J. Zhu, P. He, M.R. Lyu, Experience report: system log analysis for anomaly detection, in: 2016 IEEE 27th International Symposium on Software Reliability Engineering, ISSRE, IEEE, 2016, pp. 207–218, http://dx.doi.org/10.1109/ISSRE.2016.21.

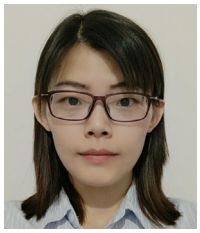
[3] S. Khan, S. Parkinson, Eliciting and utilising knowledge for security event log analysis: An association rule mining and automated planning approach, Expert Syst. Appl. 113 (2018) 116–127, http://dx.doi.org/10.1016/j.eswa.2018.07.006.

[4] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun, et al., Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs, in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19, vol. 7, 2019, pp. 4739–4745, http://dx.doi.org/10.24963/ijcai.2019/658.

[5] P. Gao, X. Xiao, D. Li, Z. Li, K. Jee, Z. Wu, C.H. Kim, S.R. Kulkarni, P. Mittal, SAQL: A stream-based query system for real-time abnormal system behavior detection, in: 27th USENIX Security Symposium, USENIX Security 18, 2018, pp. 639–656.

[6] P. Gao, X. Xiao, Z. Li, F. Xu, S.R. Kulkarni, P. Mittal, AIQL: Enabling efficient attack investigation from system monitoring data, in: 2018 USENIX Annual Technical Conference, USENIX ATC 18, 2018, pp. 113–126.
[7] T. Hacker, R. Pais, C. Rong, A Markov random field based approach for analyzing supercomputer system logs, IEEE Trans. Cloud Comput. 7 (3) (2019) 611–624, http://dx.doi.org/10.1109/TCC.2017.2678473.
[8] W. Xu, L. Huang, A. Fox, D. Patterson, M. Jordan, Online system problem detection by mining patterns of console logs, in: 2009 Ninth IEEE International Conference on Data Mining, IEEE, 2009, pp. 588–597, http://dx.doi.org/10.1109/ICDM.2009.19.
[9] J.G. Lou, Q. Fu, S. Yang, Y. Xu, J. Li, Mining invariants from console logs for system problem detection, in: USENIX Annual Technical Conference, 2010, pp. 23–25.
[10] C. De Boom, S. Van Canneyt, T. Demeester, B. Dhoedt, Representation learning for very short texts using weighted word embedding aggregation, Pattern Recognit. Lett. 80 (2016) 150–156, http://dx.doi.org/10.1016/j.patrec.2016.06.012.
[11] C. Tian, W. Rong, Y. Ouyang, Z. Xiong, Improving word representation with word pair distributional asymmetry, in: 2018 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery, CyberC, 2018, pp. 72–723, http://dx.doi.org/10.1109/CyberC.2018.00024.
[12] S. Zhao, Z. Xu, L. Liu, M. Guo, Towards accurate deceptive opinion spam detection based on word order-preserving CNN, Math. Probl. Eng. 2018 (1) (2018) 2410206, http://dx.doi.org/10.1155/2018/2410206.
[13] N. Aussel, Y. Petetin, S. Chabridon, Improving performances of log mining for anomaly prediction through NLP-based log parsing, in: 2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems, MASCOTS, 2018, http://dx.doi.org/10.1109/MASCOTS.2018.00031.
[14] B. Singh, N. Kushwaha, O.P. Vyas, A feature subset selection technique for high dimensional data using symmetric uncertainty, J. Data Anal. Inform. Process. 02 (4) (2014) 95–105, http://dx.doi.org/10.4236/jdaip.2014.24012.
[15] W. Liu, X. Liu, X. Di, B. Cai, FastLogSim: A quick log pattern parser scheme based on text similarity, in: The 13th International Conference on Knowledge Science, Engineering and Management, KSEM, 2020, pp. 211–219, http://dx.doi.org/10.1007/978-3-030-55130-8_19.
[16] W. Xu, L. Huang, A. Fox, D. Patterson, M.I. Jordan, Detecting large-scale system problems by mining console logs, in: Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, ACM, 2009, pp. 117–132.
[17] A. Oliner, J. Stearley, What supercomputers say: A study of five system logs, in: 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN'07, 2007, pp. 575–584, http://dx.doi.org/10.1109/DSN.2007.103.
[18] S. Wu, S. Wang, Parameter-free anomaly detection for categorical data, in: Proceedings of the 7th International Conference on Machine Learning and Data Mining in Pattern Recognition, MLDM'11, Springer-Verlag, Berlin, Heidelberg, 2011, pp. 112–126.
[19] A. Taha, A.S. Hadi, Anomaly detection methods for categorical data: A review, ACM Comput. Surv. 52 (2) (2019) http://dx.doi.org/10.1145/3312739.
[20] C. Callegari, S. Giordano, M. Pagano, Entropy-based network anomaly detection, in: 2017 International Conference on Computing, Networking and Communications, ICNC, 2017, pp. 334–340, http://dx.doi.org/10.1109/ICCNC.2017.7876150.
[21] Y. Zhong, W. Chen, Z. Wang, Y. Chen, K. Wang, Y. Li, X. Yin, X. Shi, J. Yang, K. Li, HELAD: A novel network anomaly detection model based on heterogeneous ensemble learning, Comput. Netw. 169 (2020) 107049, http://dx.doi.org/10.1016/j.comnet.2019.107049.
[22] R.K. Gunupudi, M. Nimmala, N. Gugulothu, S.R. Gali, CLAPP: A self constructing feature clustering approach for anomaly detection, Future Gener. Comput. Syst. 74 (2017) 417–429, http://dx.doi.org/10.1016/j.future.2016.12.040.
[23] S. Carta, A.S. Podda, D.R. Recupero, R. Saia, A local feature engineering strategy to improve network anomaly detection, Future Internet 12 (10) (2020) http://dx.doi.org/10.3390/fi12100177.
[24] R. Saia, S. Carta, R.D. Recupero, A probabilistic-driven ensemble approach to perform event classification in intrusion detection system, KDIR (2018) 139–146.
[25] S. Garg, K. Kaur, S. Batra, G. Kaddoum, N. Kumar, A. Boukerche, A multi-stage anomaly detection scheme for augmenting the security in IoT-enabled applications, Future Gener. Comput. Syst. 104 (2020) 105–118, http://dx.doi.org/10.1016/j.future.2019.09.038.
[26] A. Brown, A. Tuor, B. Hutchinson, N. Nichols, Recurrent neural network attention mechanisms for interpretable system log anomaly detection, in: Proceedings of the First Workshop on Machine Learning for Computing Systems, ACM, 2018, pp. 1–8.
[27] R. Vinayakumar, K. Soman, P. Poornachandran, Long short-term memory based operation log anomaly detection, in: 2017 International Conference on Advances in Computing, Communications and Informatics, ICACCI, IEEE, 2017, pp. 236–242.
[28] Z. Huang, W. Xu, K. Yu, Bidirectional LSTM-CRF models for sequence tagging, 2015, ArXiv preprint arXiv:1508.01991.
[29] M. Du, F. Li, G. Zheng, V. Srikumar, Deeplog: Anomaly detection and diagnosis from system logs through deep learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, ACM, 2017, pp. 1285–1298, http://dx.doi.org/10.1145/3133956.3134015.
[30] X. Zhang, Y. Xu, Q. Lin, B. Qiao, H. Zhang, Y. Dang, C. Xie, X. Yang, Q. Cheng, Z. Li, et al., Robust log-based anomaly detection on unstable log data, in: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ACM, 2019, pp. 807–817.
[31] R. Yang, D. Qu, Y. Gao, Y. Qian, Y. Tang, nLSALog: An anomaly detection framework for log sequence in security management, IEEE Access 7 (2019) 181152–181164, http://dx.doi.org/10.1109/ACCESS.2019.2953981.
[32] B. Debnath, M. Solaimani, M. Gulzar, N. Arora, C. Lumezanu, J. Xu, B. Zong, H. Zhang, G. Jiang, L. Khan, LogLens: A Real-time log analysis system, in: 2018 IEEE 38th International Conference on Distributed Computing Systems, ICDCS, IEEE Computer Society, 2018, pp. 1052–1062, http://dx.doi.org/10.1109/ICDCS.2018.00105.
[33] M. Du, F. Li, Spell: Streaming parsing of system event logs, in: 2016 IEEE 16th International Conference on Data Mining, ICDM, IEEE, 2016, pp. 859–864, http://dx.doi.org/10.1109/ICDM.2016.0103.
[34] L. You, Y. Li, Y. Wang, J. Zhang, Y. Yang, A deep learning-based RNNs model for automatic security audit of short messages, in: 2016 16th International Symposium on Communications and Information Technologies, ISCIT, 2016, pp. 225–229, http://dx.doi.org/10.1109/ISCIT.2016.7751626.
[35] J. Wang, Y. Tang, S. He, C. Zhao, P. Sharma, O. Alfarraj, A. Tolba, LogEvent2vec: LogEvent-To-vector based anomaly detection for large-scale logs in internet of things, Sensors 20 (2020) 2451, http://dx.doi.org/10.3390/s20092451.
[36] S. Yen, M. Moh, T. Moh, CausalConvLSTM: Semi-supervised log anomaly detection through sequence modeling, in: 2019 18th IEEE International Conference on Machine Learning and Applications, ICMLA, 2019, pp. 1334–1341, http://dx.doi.org/10.1109/ICMLA.2019.00217.
[37] Q. Le, T. Mikolov, Distributed representations of sentences and documents, in: Proceedings of the 31st International Conference on International Conference on Machine Learning, ICML'14, vol. 32, JMLR.org, 2014, pp. 1188–1196, http://dx.doi.org/10.5555/3044805.3045025.
[38] Y. Yuan, H. Anu, W. Shi, B. Liang, B. Qin, Learning-based anomaly cause tracing with synthetic analysis of logs from multiple cloud service components, in: 2019 IEEE 43rd Annual Computer Software and Applications Conference, COMPSAC, vol. 1, 2019, pp. 66–71, http://dx.doi.org/10.1109/COMPSAC.2019.00019.
[39] K.W. Church, Word2vec, Nat. Lang. Eng. 23 (1) (2017) 155–162.
[40] A. Joulin, E. Grave, P. Bojanowski, M. Douze, H. Jégou, T. Mikolov, Fasttext.zip: Compressing text classification models, 2016, ArXiv preprint arXiv:1612.03651.
[41] J. Pennington, R. Socher, C.D. Manning, Glove: Global vectors for word representation, in: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP, 2014, pp. 1532–1543, http://dx.doi.org/10.3115/v1/D14-1162.
[42] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, vol. 1 (Long and Short Papers), 2019, pp. 4171–4186, http://dx.doi.org/10.18653/v1/N19-1423.
[43] X. Liu, Y. Tong, A. Xu, R. Akkiraju, Using language models to pre-train features for optimizing information technology operations management tasks, in: The International Workshop on Artificial Intelligence for IT Operations, 2020.
[44] R. Yang, D. Qu, Y. Qian, Y. Dai, S. Zhu, An online log template extraction method based on hierarchical clustering, EURASIP J. Wireless Commun. Networking 2019 (135) (2019) 882–895, http://dx.doi.org/10.1186/s13638-019-1430-4.
[45] W. Meng, Y. Liu, Y. Huang, S. Zhang, F. Zaiter, B. Chen, D. Pei, A semantic-aware representation framework for online log analysis, in: 2020 29th International Conference on Computer Communications and Networks, ICCCN, 2020, pp. 1–7, http://dx.doi.org/10.1109/ICCCN49398.2020.9209707.
[46] R. Chen, S. Zhang, D. Li, Y. Zhang, F. Guo, W. Meng, D. Pei, Y. Zhang, X. Chen, Y. Liu, LogTransfer: Cross-system log anomaly detection for software systems with transfer learning, in: 2020 IEEE 31st International Symposium on Software Reliability Engineering, ISSRE, 2020, pp. 37–47, http://dx.doi.org/10.1109/ISSRE5003.2020.00013.
[47] M. Hahn, D. Jurafsky, R. Futrell, Universals of word order reflect optimization of grammars for efficient communication, Proc. Natl. Acad. Sci. 117 (5) (2020) 2347–2353, http://dx.doi.org/10.1073/pnas.1910923117.
[48] S. Zhang, D. Li, Y. Sun, W. Meng, Y. Zhang, Y. Zhang, Y. Liu, D. Pei, Unified anomaly detection for syntactically diverse logs in cloud datacenter, J. Comput. Res. Dev. 57 (4) (2020) 778–790, http://dx.doi.org/10.7544/issn1000-1239.2020.20190875.

[49] G. Prato, M. Duchesneau, S. Chandar, A. Tapp, Towards lossless encoding of sentences, in: Meeting of the Association for Computational Linguistics, 2019, http://dx.doi.org/10.18653/v1/P19-1153.
[50] P. He, J. Zhu, S. He, J. Li, M.R. Lyu, Towards automated log parsing for large-scale log data analysis, IEEE Trans. Dependable Secure Comput. 15 (6) (2018) 931–944, http://dx.doi.org/10.1109/TDSC.2017.2762673.
[51] J. Breier, J. Branišová, Anomaly detection from log files using data mining techniques, in: K.J. Kim (Ed.), Information Science and Applications, Springer Berlin Heidelberg, Berlin, Heidelberg, 2015, pp. 449–457, http://dx.doi.org/10.1007/978-3-662-46578-3_53.
[52] S.M. Erfani, S. Rajasegarar, S. Karunasekera, C. Leckie, High-dimensional and large-scale anomaly detection using a linear one-class SVM with deep learning, Pattern Recognit. 58 (2016) 121–134, http://dx.doi.org/10.1016/j.patcog.2016.03.028.
[53] X. Xie, Z. Jin, J. Wang, L. Yang, Y. Lu, T. Li, Confidence guided anomaly detection model for anti-concept drift in dynamic logs, J. Netw. Comput. Appl. 162 (2020) 102659, http://dx.doi.org/10.1016/j.jnca.2020.102659.
[54] L. Bao, Q. Li, P. Lu, J. Lu, T. Ruan, K. Zhang, Execution anomaly detection in large-scale systems through console log analysis, J. Syst. Softw. 143 (2018) 172–186, http://dx.doi.org/10.1016/j.jss.2018.05.016.
[55] J. Yang, L. Wang, Z. Xu, A novel semantic-aware approach for detecting malicious web traffic, in: International Conference on Information and Communications Security, Springer, Springer International Publishing, 2018, pp. 633–645, http://dx.doi.org/10.1007/978-3-319-89500-0_54.
[56] Q. Fu, J.-G. Lou, Y. Wang, J. Li, Execution anomaly detection in distributed systems through unstructured log analysis, in: Proceedings of the 2009 Ninth IEEE International Conference on Data Mining, ICDM '09, IEEE Computer Society, USA, 2009, pp. 149–158, http://dx.doi.org/10.1109/ICDM.2009.60.
[57] H. Studiawan, F. Sohel, C. Payne, Automatic event log abstraction to support forensic investigation, in: Proceedings of the Australasian Computer Science Week Multiconference, ACSW '20, Association for Computing Machinery, New York, NY, USA, 2020, http://dx.doi.org/10.1145/3373017.3373018.
[58] D. Kingma, J. Ba, Adam: A method for stochastic optimization, Intl. Conf. Learn. Represent. (2014).
[59] T. Li, J. Ma, C. Sun, Dlog: diagnosing router events with syslogs for anomaly detection, J. Supercomput. 74 (2) (2018) 845–867, http://dx.doi.org/10.1007/s11227-017-2165-9.
[60] J. Hu, H. Yang, M.R. Lyu, I. King, A. Man-Cho So, Online nonlinear AUC maximization for imbalanced data sets, IEEE Trans. Neural Netw. Learn. Syst. 29 (4) (2018) 882–895, http://dx.doi.org/10.1109/TNNLS.2016.2610465.
[61] W. Liu, Testing statistical hypotheses of equivalence and noninferiority, 2nd edn by Stefan Wellek, Austral. New Zealand J. Stat. 53 (1) (2011) 126–127, http://dx.doi.org/10.1111/j.1467-842X.2011.00598.x.
[62] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, Springer Science & Business Media, 2012.



Xu Liu received B.E. degree in Network Engineering from Qingdao University in 2013. She received M.E. degree in Computer Software Theory from Changchun University of Science and Technology. She is currently a doctoral student in computer science and technology at the Changchun University of Science and Technology, China. Her research interests include network security, information security, anomaly detection, artificial intelligence, big data, game theory and cloud computing.

Weiyou Liu received B.E. degree in Communication Engineering from Changchun University of Science and Technology in 2018. He is currently a master student in computer science and technology at the Changchun University of Science and Technology, China. His research interests include network security, big data, information security, anomaly detection and artificial intelligence.

Xiaoqiang Di received B.S. degree in computer science and technology from Changchun University of Science and Technology in 2002, and M.S. and Ph.D. degrees in communication and information systems from Changchun University of Science and Technology in 2007 and 2014, respectively. He was a visiting scholar at Norwegian University of Science and Technology, Norway, from Aug. 2012 to Aug. 2013. He is currently a professor and supervisor of Ph.D. in Changchun University of Science and Technology. His major research interests include network information security and integrated network.

Jinqing Li received B.S. degree from Changchun University of Technology in 2002, and M.S. and Ph.D. degrees from Changchun University of Science and Technology in 2007 and 2014, respectively. She is currently an associate professor and a master student supervisor in Changchun University of Science and Technology. Her research interests are in the areas of network security, information security, and image encryption.

Binbin Cai received B.E. degree from ChangChun University of Technology in 2003, and M.S. degree from ChangChun University of Science and Technology in 2010, respectively. He is currently an engineer of Information Center in ChangChun University of Science and Technology. His research interests are network monitor, security maintenance, big data and information security.

Weiwu Ren received his master's degree in network and information security from Jilin University in 2010 and his doctor's degree in computer system architecture from Jilin University in 2013. His main works includes intrusion detection, machine learning and knowledge engineering. He is a membership of China computer federation.

Huamin Yang received his B.E. degree from Dalian University of Technology in 1985, China. He received M.E. degree from JiLin University. He received Ph.D. from Changchun University of Science and Technology in 2002. He is currently a professor and supervisor of Ph.D. in Changchun University of Science and Technology. Field of research: information security, computer simulation modeling and virtual reality technology.